Rapport Labo 3 - Computer Architectuur

Victor Deforce en Cían Olieslagers

March 2, 2025

1 Probleemstelling

1.1 Opgave

1. Warps
   Implementeer een kernel die een afbeelding inverteert volgens de formule $x = 255 - x$. Onderzoek hoe het aantal threads de rekentijd beïnvloeden.

2. Thread-divergentie en Gaussian blur
   Implementeer een kernel die een afbeelding verwerkt met een aangepaste aanpak:

   - De groene en blauwe waarden worden verwerkt met de originele inversiekernel ($x = 255 - x$).
   - De rode waarden worden verwerkt met een Gaussian blur-kernel, volgens de volgende formule:
     $$r_i = 0.1 \cdot r_{i-2} + 0.25 \cdot r_{i-1} + 0.5 \cdot r_i + 0.25 \cdot r_{i+1} + 0.1 \cdot r_{i+2}$$

   Onderzoek hoe thread-divergentie de rekentijd beïnvloedt.

2 Implementatie

2.1 Deel 1: Warp

De CUDA-kernel ‘InvertImage‘ voert de inversie uit, waarbij elke thread één pixel bewerkt volgens de formule:

$$x = 255 - x$$

Hierbij wordt de intensiteit van elke pixel omgekeerd. Het aantal blokken en threads wordt berekend op basis van de afbeeldingsgrootte, met een variërend aantal threads per blok. De kernel voorkomt out-of-bounds access door een indexcontrole.
Om de verwerking op de GPU uit te voeren, wordt geheugen gealloceerd op zowel de host (CPU) als de device (GPU). De originele afbeelding wordt gekopieerd naar het GPU-geheugen, waarna de CUDA-kernel de bewerking uitvoert. Na verwerking wordt het resultaat terug naar de CPU gekopieerd en als PPM-bestand opgeslagen.

2.2 Deel 2: Thread divergence

De **CUDA-kernel** `invertKernel` voert zowel een **kleurinversie** als een **Gaussian blur** uit, waarbij elke thread één pixel bewerkt. Het onderscheid tussen de kleurkanalen wordt gemaakt door te controleren of de index (`idx`) deelbaar is door 3:

- **Groene en blauwe pixels** worden geïnverteerd volgens de formule:

$$x = 255 - x$$

  Dit gebeurt wanneer `idx % 3 != 0`, wat betekent dat de index niet overeenkomt met een rood kanaal.

- **Rode pixels** worden verwerkt met een **1D-Gaussian blur**, waarbij elke rode pixel wordt berekend als een gewogen som van zijn buren:

$$r_i = 0.1 \cdot r_{i-2} + 0.25 \cdot r_{i-1} + 0.5 \cdot r_i + 0.25 \cdot r_{i+1} + 0.1 \cdot r_{i+2}$$

  Dit gebeurt wanneer `idx % 3 == 0`, wat aangeeft dat de pixel zich in het rode kanaal bevindt.

De geheugenallocatie en de blokberekening blijven identiek aan de vorige oefening. Omdat de rode pixels meer berekeningen vereisen dan de groene en blauwe pixels, ontstaat er **thread divergence**, wat kan leiden tot prestatieverlies binnen de GPU.

Ter vergelijking implementeren we ook een versie waar eerst alle subpixels per kleur worden gegroepeerd. De Gausian blur wordt hier toegepast op de pixels waarvoor geld: $idx < size/3$

3 Observatie

3.1 Tijd vergelijking variërende thread per blok verhouding

We laten het programma voor verschillende thread per block verhoudingen 1000 keer uitvoeren. De verhouding start aan 1024 en halveert tot aan 1. Als we kijken naar de grafiek aan het einde van het document, zien we dat de verwerkingstijd relatief constant blijft tot en met een threadblokgrootte van 64. Vanaf thread 32 en kleiner neemt de verwerkingstijd duidelijk toe. Dit heeft te maken met hoe warps binnen CUDA werken.

Een warp bestaat uit 32 threads, wat betekent dat wanneer een threadblok minder dan 32 threads bevat, er nog steeds een volledige warp wordt aangemaakt. Als we bijvoorbeeld 16 threads per blok hebben, blijft de helft van de warp ongebruikt. Hierdoor lijkt thread 16 korter, omdat de overige 16 threads inactief zijn en geen extra berekeningen uitvoeren.

Bij nog kleinere threadaantallen (bijvoorbeeld 8, 4, 2, 1) wordt de efficiëntie nog verder verlaagd, omdat slechts een fractie van een warp nuttig werk doet, terwijl de rest van de threads wacht. Dit verklaart de sterke stijging in verwerkingstijd.

We zien ook al een kleine stijging in tijd bij 64 threads per block. Dit is te verklaren doordat eens alle threads binnen een block afgerond zijn er een nieuwe block opgestart moet worden. Hiermee gaat een zekere overhead gepaard.

3.2 Effect van thread divergence

Doordat voor elke rode rode subpixel meer werk moet gedaan worden en de subpixels gemengd zijn binnen een warp, heeft elke warp evenveel tijd nodig als de traagste thread. Dit zien we in de resultaten in fig 1. De looptijd is een factor kleiner als alle threads die op de rode subpixels inwerken samen gegroepeerd worden in warps ten op zichtte van ze mengen met de andere threads. Er gaat dus duidelijk tijd verloren door idle threads die cores innemen maar ze niet ten volle benutten.

4 Conclusie

De keuzes die men maakt bij het verdelen van taken over verschillende threads en het groeperen van threads in verschillende blocks, hebben een grote invloed op de looptijd van het programma. Het is dus aangewezen om een veelvoud van 32 threads per block te definiëren, zodat steeds volledige gevulde warps gescheduled kunnen worden. Verder groepeert men best threads met dezelfde complexiteit in dezelfde warps om te verzekeren dat ze slechts kort op elkaar moeten wachten en er dus geen cores worden ingenomen door idle threads.

5 Appendix - code

5.1 Deel 1: Warps

```
#define M 960       // VR width
#define N 1280       // VR height
#define C 3         // Colors
#define OFFSET 16   // Header length


__global__ void InvertImage(uint8_t* image_out, uint8_t* image_in, int size)
{
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (idx < size) {  // Prevent out-of-bounds access
        image_out[idx] = 255 - image_in[idx];
    }

}
uint8_t* get_image_array(void) {
    FILE* imageFile;
    imageFile = fopen("./anna.ppm", "rb");
    if (imageFile == NULL) {
        perror("ERROR: Cannot open output2 file");
        exit(EXIT_FAILURE);
    }

    // Initialize empty image array
    uint8_t* image_array = (uint8_t*)malloc(M * N * C * sizeof(uint8_t) + OFFSET);

    // Read the image
    fread(image_array, sizeof(uint8_t), M * N * C * sizeof(uint8_t) + OFFSET, imageFile);

    // Close the file
    fclose(imageFile);

    // Move the starting pointer and return the flattened image array
    return image_array + OFFSET;
}

void save_image_array(uint8_t* image_array) {
    FILE* imageFile;
    imageFile = fopen("./output_image.ppm", "wb");
    if (imageFile == NULL) {
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }
    // Configure the file
    fprintf(imageFile, "P6\n");               // P6 filetype
    fprintf(imageFile, "%d %d\n", M, N);      // dimensions
    fprintf(imageFile, "255\n");              // Max pixel
    // Write the image
    fwrite(image_array, 1, M * N * C, imageFile);
    // Close the file
    fclose(imageFile);
}
```

```
int main()
{
    // Read the image
    uint8_t* image_array = get_image_array();

    // Allocate output
    uint8_t* new_image_array = (uint8_t*)malloc(M * N * C);
    uint8_t* h_image_in = get_image_array();
    uint8_t* h_image_out = (uint8_t*)malloc(M * N * C);
    uint8_t* d_image_in, * d_image_out;
    int size = M * N * C;
    cudaMalloc((void**)&d_image_in, size);
    cudaMalloc((void**)&d_image_out, size);
    cudaMemcpy(d_image_in, h_image_in, size, cudaMemcpyHostToDevice);
    int threads_per_block = 256;
    int num_blocks = (size + threads_per_block - 1) / threads_per_block;
    auto start = std::chrono::high_resolution_clock::now();

    //oef1
    InvertImage <<<num_blocks, threads_per_block >>> (d_image_out, d_image_in, size);
    cudaDeviceSynchronize();
    auto end = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> elapsed = end - start;
    std::cout << "Verwerkingstijd: " << elapsed.count() << " seconden" << std::endl;

    cudaMemcpy(h_image_out, d_image_out, size, cudaMemcpyDeviceToHost);

    // Opslaan van de afbeelding
    save_image_array(h_image_out);

    // Opschonen
    cudaFree(d_image_in);
    cudaFree(d_image_out);
    free(h_image_out);
    return 0;
}
```

5.2 Deel 2: Thread divergence

De main is hetzelfde, enkel de kernel hebben we anders geïmplementeerd.

```
 __global__ void invertKernel(uint8_t *old_array, uint8_t *new_array) {
    int idx = threadIdx.x + blockDim.x * blockIdx.x;
    if(idx%3!=0){
        new_array[idx] = 255 - old_array[idx];
    } else {
        new_array[idx] = 0.1*old_array[idx-6] +
        0.25*old_array[idx-3] + 0.5*old_array[idx] +
        0.25*old_array[idx+3] + 0.1*old_array[idx+6];
    }
}
```

5.2.1 Grouped subpixels

```
    __global__ void antiDivergenceKernel(uint8_t *old_array, uint8_t *new_array, int MN) {
  int idx = threadIdx.x + blockDim.x * blockIdx.x;
  if(idx > MN){
      new_array[idx] = 255 - old_array[idx];
  } else {
      new_array[idx] = 0.1*old_array[idx-2] + 0.25*old_array[idx-1] + 0.5*old_array[idx] + 0.25*old_a
  }
}

 int main(void) {
   FILE *file = file = fopen("output_lab3.csv", "w");
   if (!file) {
     perror("File opening failed");
     exit(EXIT_FAILURE);
   } else {
     fprintf(stderr, "Writing output to file %s\n", "output.txt");
   }
   uint8_t *image_array = get_image_array();
   /* for (int k = 1; k <= 1024; k = k * 2) { // Read the image*/
     int iterations = 100;
     for (int j = 0; j < iterations; j++) {
       // Allocate output
       int k = 1;
       uint8_t *new_image_array = (uint8_t *)malloc(M * N * C);
       uint8_t *reordered_image_array = (uint8_t *)malloc(M * N * C);
       for(int i = 0; i< M*N; i++){
        reordered_image_array[i]= image_array[3*i];
        reordered_image_array[i+M*N]= image_array[3*i+1];
        reordered_image_array[i+2*M*N]= image_array[3*i+2];
       }
       uint8_t *new_image_array_gpu;
       uint8_t *image_array_gpu;
       int nBlocks = M * N * C / 1024;
       int nThreads = 1024;

       cudaMalloc((void **)&image_array_gpu, M * N * C * sizeof(uint8_t));
       cudaMalloc((void **)&new_image_array_gpu, M * N * C * sizeof(uint8_t));
       cudaMemcpy(image_array_gpu, reordered_image_array, M * N * C * sizeof(uint8_t),
                  cudaMemcpyHostToDevice);
```

```
      auto start = std::chrono::steady_clock::now();

      antiDivergenceKernel<<<nBlocks * k, nThreads / k>>>(image_array_gpu,
                                                  new_image_array_gpu, M*N);
      cudaDeviceSynchronize();

      long duration = std::chrono::duration_cast<std::chrono::microseconds>(
                          std::chrono::steady_clock::now() - start)
                          .count();

      cudaMemcpy(new_image_array, new_image_array_gpu,
                 M * N * C * sizeof(uint8_t), cudaMemcpyDeviceToHost);
      cudaFree(image_array_gpu);
      cudaFree(new_image_array_gpu);

      fprintf(file, "%lu, ", duration);

      uint8_t *new_reordered_image_array = (uint8_t *)malloc(M * N * C);
      for(int i = 0; i< M*N; i++){
       new_reordered_image_array[3*i] = new_image_array[i];
       new_reordered_image_array[3*i+1] = new_image_array[i+M*N];
       new_reordered_image_array[3*i+2] = new_image_array[i+2*M*N];
      }

      // Save the image
      save_image_array(new_reordered_image_array);
      free(new_image_array);
      free(new_reordered_image_array);
    }
    fprintf(file, "\n");
  //}

  return 0;
}
```

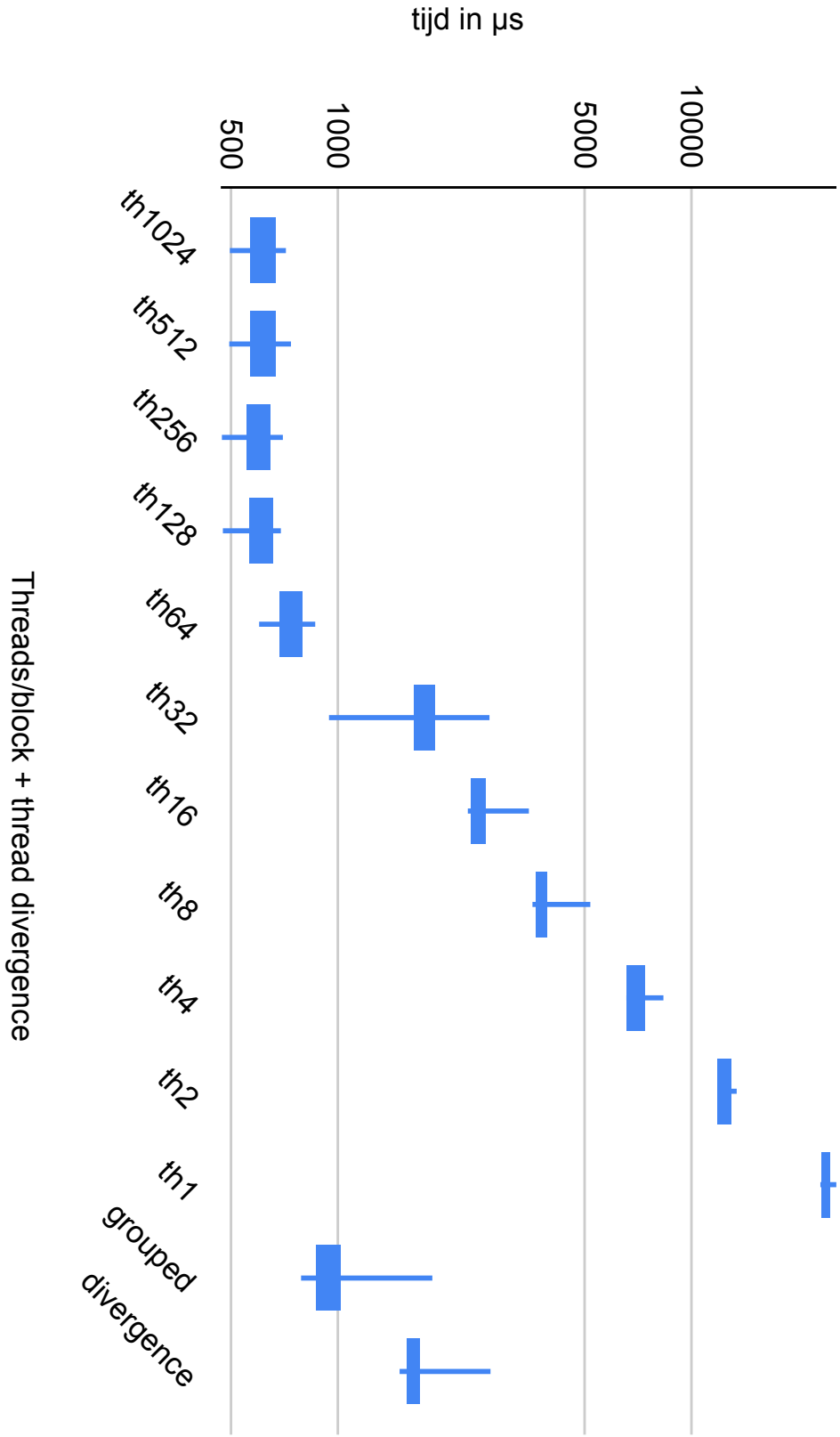


Figure 1: Meetwaarden